MySQL Practice Sheet — Advanced

Database: marketdb · 75 Practical Queries · Combinations & Complex Analysis

● EASY · ▲ MEDIUM · ■ HARD | Includes: Joins + Aggregation, Subqueries, CASE/Window Functions, Complex Patterns

Contents

- Section 1: Joins + Aggregation (Q1-Q10)
- Section 2: Subqueries + Aggregation (Q11-Q20)
- Section 3: CASE WHEN + GROUP BY (Q21-Q30)
- Section 4: Window Functions (Q31-Q45)
- Section 5: Complex Combinations (Q46-Q75)

JOINS + AGGREGATION

●	**Q1. List all sales records with customer name, product name, and total sales amount per transaction. Join fact_sales, dim_cust, and dim_prod tables.**

✔ SOLUTION

```
SELECT f.ord_id, c.cust_name, p.prod_name, f.sales
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
ORDER BY f.sales DESC;
```

■ *Use INNER JOIN to combine data from three dimension tables with the fact table.*

●	**Q2. Display each product with its name, category, total sales count, and total profit. Include unsold products.**

✔ SOLUTION

```
SELECT p.prod_id, p.prod_name, p.category,
       COUNT(f.ord_id) AS total_sales,
       COALESCE(ROUND(SUM(f.profit), 2), 0) AS total_profit
FROM dim_prod p
LEFT JOIN fact_sales f ON p.prod_id = f.prod_id
GROUP BY p.prod_id, p.prod_name, p.category
ORDER BY total_profit DESC;
```

■ *Use LEFT JOIN to keep all products and COUNT/SUM for aggregation.*

●	**Q3. Show each customer with their total number of orders, total sales amount, and average order value.**

✔ SOLUTION

```
SELECT c.cust_id, c.cust_name,
```

```
       COUNT(DISTINCT f.ord_id) AS total_orders,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.sales) / COUNT(DISTINCT f.ord_id), 2) AS avg_order_value
FROM dim_cust c
LEFT JOIN fact_sales f ON c.cust_id = f.cust_id
GROUP BY c.cust_id, c.cust_name
ORDER BY total_sales DESC;
```

■ *Use COUNT(DISTINCT ord_id) to get unique orders and division for average.*

▲ Q4. For each customer segment, display the segment name, number of distinct customers, total sales, and average profit per customer.

✔ SOLUTION

```
SELECT c.cust_segment,
       COUNT(DISTINCT c.cust_id) AS num_customers,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.profit) / COUNT(DISTINCT c.cust_id), 2) AS
avg_profit_per_customer
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
GROUP BY c.cust_segment
ORDER BY total_sales DESC;
```

■ *Group by segment and use COUNT(DISTINCT cust_id) to avoid double-counting.*

▲ Q5. Find each product category with category name, total number of unique customers, total profit, and profit margin (profit/sales * 100).

✔ SOLUTION

```
SELECT p.category,
       COUNT(DISTINCT f.cust_id) AS unique_customers,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.profit) / SUM(f.sales) * 100, 2) AS profit_margin_pct
FROM fact_sales f
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY p.category
ORDER BY total_profit DESC;
```

■ *Calculate margin using SUM aggregates, multiply by 100 for percentage.*

▲ Q6. List all orders with order date, total line items, total order profit, and shipping mode. Join four tables.

✔ SOLUTION

```
SELECT d.ord_id, d.order_date,
       COUNT(f.prod_id) AS line_items,
       ROUND(SUM(f.profit), 2) AS order_profit,
       s.ship_mode
FROM fact_sales f
```

```
INNER JOIN dim_order d ON f.ord_id = d.ord_id
INNER JOIN dim_shipping s ON f.ship_id = s.ship_id
GROUP BY d.ord_id, d.order_date, s.ship_mode
ORDER BY order_profit DESC;
```

■ *Group by ord_id and aggregate line items/profit at order level.*

▲	**Q7. Show each shipping mode with average order value, average shipping days (ship_date - order_date), and total profit.**

✔ SOLUTION

```
SELECT s.ship_mode,
       ROUND(AVG(f.sales), 2) AS avg_order_value,
       ROUND(AVG(DATEDIFF(d.ship_date, d.order_date)), 2) AS avg_ship_days,
       ROUND(SUM(f.profit), 2) AS total_profit
FROM fact_sales f
INNER JOIN dim_shipping s ON f.ship_id = s.ship_id
INNER JOIN dim_order d ON f.ord_id = d.ord_id
GROUP BY s.ship_mode
ORDER BY total_profit DESC;
```

■ *Use DATEDIFF for date difference and AVG for average metrics.*

■	**Q8. For each customer, display total orders, total sales, total profit, and profit rank (highest to lowest). Only include customers with profit > 0.**

✔ SOLUTION

```
SELECT c.cust_id, c.cust_name,
       COUNT(DISTINCT f.ord_id) AS total_orders,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.profit), 2) AS total_profit,
       DENSE_RANK() OVER (ORDER BY SUM(f.profit) DESC) AS profit_rank
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
GROUP BY c.cust_id, c.cust_name
HAVING SUM(f.profit) > 0
ORDER BY profit_rank;
```

■ *Use DENSE_RANK() window function and HAVING for filtering after grouping.*

■	**Q9. List products by category showing top product name (by profit) in each category, category profit, and product profit percentage.**

✔ SOLUTION

```
WITH cat_profits AS (
  SELECT p.category, p.prod_name, SUM(f.profit) AS prod_profit,
         ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(f.profit)
DESC) AS rn
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
```

```
  GROUP BY p.category, p.prod_name
)
SELECT category, prod_name, prod_profit,
       ROUND(prod_profit / (SELECT SUM(f.profit) FROM fact_sales f
                            JOIN dim_prod p ON f.prod_id = p.prod_id
                            WHERE p.category = cat_profits.category) * 100, 2) AS
pct_of_cat
FROM cat_profits
WHERE rn = 1
ORDER BY prod_profit DESC;
```

■ *Use ROW_NUMBER() with PARTITION BY to get top product per category.*

■	**Q10. Show customer segment analysis: segment name, customer count, total orders, total sales, total profit, and profit per order.**

✔ SOLUTION

```
SELECT c.cust_segment,
       COUNT(DISTINCT c.cust_id) AS customer_count,
       COUNT(DISTINCT f.ord_id) AS total_orders,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.profit) / COUNT(DISTINCT f.ord_id), 2) AS profit_per_order
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
GROUP BY c.cust_segment
ORDER BY total_profit DESC;
```

■ *Count both distinct customers and orders for segment metrics.*

SUBQUERIES + AGGREGATION

•	**Q11. Find all products with sales above the average product sales amount.**

✔ SOLUTION

```
SELECT p.prod_id, p.prod_name,
       ROUND(SUM(f.sales), 2) AS total_sales
FROM dim_prod p
INNER JOIN fact_sales f ON p.prod_id = f.prod_id
GROUP BY p.prod_id, p.prod_name
HAVING SUM(f.sales) > (
  SELECT AVG(prod_sales)
  FROM (SELECT SUM(sales) AS prod_sales FROM fact_sales GROUP BY prod_id) sub
)
ORDER BY total_sales DESC;
```

■ *Use subquery to calculate average of per-product totals, then use HAVING.*

•	**Q12. List customers whose total profit is greater than the average profit across all customers.**

✔ **SOLUTION**

```
SELECT c.cust_id, c.cust_name,
       ROUND(SUM(f.profit), 2) AS total_profit
FROM dim_cust c
INNER JOIN fact_sales f ON c.cust_id = f.cust_id
GROUP BY c.cust_id, c.cust_name
HAVING SUM(f.profit) > (
  SELECT AVG(cust_profit)
  FROM (SELECT SUM(profit) AS cust_profit FROM fact_sales GROUP BY cust_id) sub
)
ORDER BY total_profit DESC;
```

■ *Derived table in subquery computes per-customer totals for averaging.*

▲	**Q13. Find top 3 product categories by total profit using subquery approach.**

✔ **SOLUTION**

```
SELECT category, total_profit
FROM (
  SELECT p.category, SUM(f.profit) AS total_profit,
         ROW_NUMBER() OVER (ORDER BY SUM(f.profit) DESC) AS rn
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category
) ranked
WHERE rn <= 3
ORDER BY total_profit DESC;
```

■ *Use subquery with ROW_NUMBER() and filter by rank.*

▲	**Q14. Display products that have been ordered more times than the average number of orders per product.**

✔ **SOLUTION**

```
SELECT prod_id,
       COUNT(*) AS order_count
FROM fact_sales
GROUP BY prod_id
HAVING COUNT(*) > (
  SELECT AVG(cnt)
  FROM (SELECT COUNT(*) AS cnt FROM fact_sales GROUP BY prod_id) sub
)
ORDER BY order_count DESC;
```

■ *Nested subquery in HAVING: inner subquery counts per product, outer takes average.*

▲	**Q15. List customers with total sales exceeding the top 10th percentile of all customer sales.**

✔ **SOLUTION**

```
SELECT cust_id,
       ROUND(SUM(sales), 2) AS total_sales
FROM fact_sales
GROUP BY cust_id
HAVING SUM(sales) >= (
  SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY cust_sales)
  FROM (SELECT SUM(sales) AS cust_sales FROM fact_sales GROUP BY cust_id) sub
)
ORDER BY total_sales DESC;
```

■ *Use PERCENTILE_CONT for statistical threshold calculation.*

▲	**Q16. Find each product category with its maximum unit price product, showing category name, product name, and unit price.**

✔ SOLUTION

```
SELECT p.category, p.prod_name, p.unit_price
FROM dim_prod p
WHERE p.unit_price = (
  SELECT MAX(p2.unit_price)
  FROM dim_prod p2
  WHERE p2.category = p.category
)
ORDER BY p.category, p.unit_price DESC;
```

■ *Correlated subquery: compares each product against max in its category.*

■	**Q17. Find orders with above-average profit, showing order details and rank by profit.**

✔ SOLUTION

```
WITH order_profits AS (
  SELECT d.ord_id, d.order_date,
         SUM(f.profit) AS order_profit,
         ROW_NUMBER() OVER (ORDER BY SUM(f.profit) DESC) AS profit_rank
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY d.ord_id, d.order_date
)
SELECT * FROM order_profits
WHERE order_profit > (SELECT AVG(order_profit) FROM order_profits)
ORDER BY profit_rank;
```

■ *CTE for clarity; subquery in WHERE to calculate overall average.*

■	**Q18. Compare each customer's average order value to the global average, showing variance.**

✔ SOLUTION

```
SELECT c.cust_id, c.cust_name,
```

```
       ROUND(AVG(f.sales), 2) AS cust_avg_order_value,
       (SELECT ROUND(AVG(sales), 2) FROM fact_sales) AS global_avg_order_value,
       ROUND(AVG(f.sales) - (SELECT AVG(sales) FROM fact_sales), 2) AS variance
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
GROUP BY c.cust_id, c.cust_name
ORDER BY variance DESC;
```

■ *Scalar subquery in SELECT for global comparison.*

■ Q19. Find products that exist in certain orders matching specific profit criteria.

✔ SOLUTION

```
SELECT DISTINCT p.prod_id, p.prod_name, p.category
FROM dim_prod p
WHERE p.prod_id IN (
  SELECT f.prod_id
  FROM fact_sales f
  WHERE f.profit > (SELECT AVG(profit) FROM fact_sales)
)
AND p.prod_id NOT IN (
  SELECT f.prod_id
  FROM fact_sales f
  WHERE f.profit < -100
)
ORDER BY p.prod_name;
```

■ *Use IN and NOT IN with subqueries for existence checks.*

■ Q20. Show which categories contribute what percentage to total profit, ordering by contribution.

✔ SOLUTION

```
WITH cat_profits AS (
  SELECT p.category,
         SUM(f.profit) AS cat_profit,
         (SELECT SUM(profit) FROM fact_sales) AS total_profit
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category
)
SELECT category,
       ROUND(cat_profit, 2) AS category_profit,
       ROUND(cat_profit / total_profit * 100, 2) AS profit_percentage
FROM cat_profits
ORDER BY profit_percentage DESC;
```

■ *Calculate total once in CTE, then compute percentages in outer query.*

CASE WHEN + GROUP BY

•	**Q21. Classify products into price tiers (Premium >200, Standard 50-200, Budget <50) and count each tier.**

✔ SOLUTION

```
SELECT
       CASE
         WHEN unit_price > 200 THEN 'Premium'
         WHEN unit_price >= 50 THEN 'Standard'
         ELSE 'Budget'
       END AS price_tier,
       COUNT(*) AS product_count,
       ROUND(AVG(unit_price), 2) AS avg_price
FROM dim_prod
GROUP BY price_tier
ORDER BY avg_price DESC;
```

■ *Use CASE WHEN and GROUP BY on the CASE expression directly.*

•	**Q22. Label each sale as 'Profitable', 'Break-even', or 'Loss' and show count and total profit per category.**

✔ SOLUTION

```
SELECT
       CASE
         WHEN f.profit > 0 THEN 'Profitable'
         WHEN f.profit = 0 THEN 'Break-even'
         ELSE 'Loss'
       END AS profit_status,
       COUNT(*) AS num_records,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.sales), 2) AS total_sales
FROM fact_sales f
GROUP BY profit_status
ORDER BY total_profit DESC;
```

■ *GROUP BY a CASE expression and aggregate within each group.*

•	**Q23. Categorize discounts as 'No Discount' (0), 'Low' (0-0.1), 'Medium' (0.1-0.2), 'High' (>0.2) and count sales.**

✔ SOLUTION

```
SELECT
       CASE
         WHEN f.discount = 0 THEN 'No Discount'
         WHEN f.discount <= 0.1 THEN 'Low'
         WHEN f.discount <= 0.2 THEN 'Medium'
         ELSE 'High'
```

```
        END AS discount_category,
        COUNT(*) AS num_sales,
        ROUND(AVG(f.sales), 2) AS avg_sales_amount
FROM fact_sales f
GROUP BY discount_category
ORDER BY num_sales DESC;
```

■ *Use nested CASE conditions with decimal comparisons.*

▲ Q24. Create a pivot showing total sales per quarter for each product category.

✔ SOLUTION

```
SELECT p.category,
        ROUND(SUM(CASE WHEN MONTH(d.order_date) BETWEEN 1 AND 3 THEN f.sales
ELSE 0 END), 2) AS Q1_sales,
        ROUND(SUM(CASE WHEN MONTH(d.order_date) BETWEEN 4 AND 6 THEN f.sales
ELSE 0 END), 2) AS Q2_sales,
        ROUND(SUM(CASE WHEN MONTH(d.order_date) BETWEEN 7 AND 9 THEN f.sales
ELSE 0 END), 2) AS Q3_sales,
        ROUND(SUM(CASE WHEN MONTH(d.order_date) BETWEEN 10 AND 12 THEN f.sales
ELSE 0 END), 2) AS Q4_sales
FROM fact_sales f
INNER JOIN dim_order d ON f.ord_id = d.ord_id
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY p.category
ORDER BY (Q1_sales + Q2_sales + Q3_sales + Q4_sales) DESC;
```

■ *Use conditional SUM(CASE) to pivot months into quarter columns.*

▲ Q25. For each customer segment, count sales by delivery speed: Fast (≤3 days), Normal (4-7), Slow (>7).

✔ SOLUTION

```
SELECT c.cust_segment,
        SUM(CASE WHEN DATEDIFF(d.ship_date, d.order_date) <= 3 THEN 1 ELSE 0
END) AS fast_count,
        SUM(CASE WHEN DATEDIFF(d.ship_date, d.order_date) BETWEEN 4 AND 7 THEN 1
ELSE 0 END) AS normal_count,
        SUM(CASE WHEN DATEDIFF(d.ship_date, d.order_date) > 7 THEN 1 ELSE 0 END)
AS slow_count
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
INNER JOIN dim_order d ON f.ord_id = d.ord_id
GROUP BY c.cust_segment
ORDER BY fast_count DESC;
```

■ *Use SUM(CASE WHEN 1 ELSE 0) for conditional counting.*

▲ Q26. Show top products by sales with tier classification (Top 10%, Next 20%, Rest).

✔ **SOLUTION**

```
WITH prod_sales AS (
  SELECT prod_id,
         SUM(sales) AS total_sales,
         PERCENT_RANK() OVER (ORDER BY SUM(sales) DESC) AS percentile
  FROM fact_sales
  GROUP BY prod_id
)
SELECT prod_id, total_sales,
       CASE
         WHEN percentile <= 0.10 THEN 'Top 10%'
         WHEN percentile <= 0.30 THEN 'Next 20%'
         ELSE 'Rest'
       END AS tier
FROM prod_sales
ORDER BY total_sales DESC;
```

■ *Use PERCENT_RANK window function with CASE for percentile-based tiers.*

■	**Q27. Calculate customer health score: 1 point if avg profit > global average, 1 if orders > 10. Classify as Gold/Silver/Bronze.**

✔ **SOLUTION**

```
WITH cust_stats AS (
  SELECT c.cust_id, c.cust_name,
         COUNT(*) AS total_orders,
         ROUND(AVG(f.profit), 2) AS avg_profit
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  GROUP BY c.cust_id, c.cust_name
),
scoring AS (
  SELECT cust_id, cust_name, total_orders, avg_profit,
         (CASE WHEN avg_profit > (SELECT AVG(profit) FROM fact_sales) THEN 1
ELSE 0 END
         + CASE WHEN total_orders > 10 THEN 1 ELSE 0 END) AS health_score
  FROM cust_stats
)
SELECT cust_id, cust_name, health_score,
       CASE
         WHEN health_score = 2 THEN 'Gold'
         WHEN health_score = 1 THEN 'Silver'
         ELSE 'Bronze'
       END AS tier
FROM scoring
ORDER BY health_score DESC;
```

■ *Combine multiple CASE conditions with addition for composite scoring.*

■	**Q28. Show profit margin analysis for categories: >30% High, 10-30% Medium, <10% Low. Include avg margin by category.**

✔ **SOLUTION**

```
WITH margin_calc AS (
  SELECT p.category,
         SUM(f.profit) AS total_profit,
         SUM(f.sales) AS total_sales,
         (SUM(f.profit) / NULLIF(SUM(f.sales), 0) * 100) AS margin_pct
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category
)
SELECT category, ROUND(margin_pct, 2) AS margin_pct,
       CASE
         WHEN margin_pct > 30 THEN 'High'
         WHEN margin_pct >= 10 THEN 'Medium'
         ELSE 'Low'
       END AS margin_class,
       ROUND(total_profit, 2) AS total_profit
FROM margin_calc
ORDER BY margin_pct DESC;
```

■ *Use NULLIF to prevent division by zero in percentage calculation.*

■ **Q29. Create revenue classification: 'Star' (profit+sales top 25%), 'Standard' (next 50%), 'Underperformer' (bottom 25%) for products.**

✔ **SOLUTION**

```
WITH prod_metrics AS (
  SELECT p.prod_id, p.prod_name,
         SUM(f.sales) + SUM(f.profit) AS revenue_metric,
         NTILE(4) OVER (ORDER BY SUM(f.sales) + SUM(f.profit)) AS quartile
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.prod_id, p.prod_name
)
SELECT prod_id, prod_name, revenue_metric,
       CASE
         WHEN quartile = 4 THEN 'Star'
         WHEN quartile IN (2, 3) THEN 'Standard'
         ELSE 'Underperformer'
       END AS classification
FROM prod_metrics
ORDER BY revenue_metric DESC;
```

■ *Use NTILE(4) to create quartiles, then CASE for classification.*

■ **Q30. For each month, show total sales, profit, and classify month as 'Growth' (vs prior) or 'Decline'.**

✔ **SOLUTION**

```
WITH monthly_totals AS (
  SELECT MONTH(d.order_date) AS month_num,
         SUM(f.sales) AS monthly_sales,
         LAG(SUM(f.sales)) OVER (ORDER BY MONTH(d.order_date)) AS prev_sales
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY MONTH(d.order_date)
)
SELECT month_num, monthly_sales,
       CASE
         WHEN prev_sales IS NULL THEN 'Baseline'
         WHEN monthly_sales > prev_sales THEN 'Growth'
         ELSE 'Decline'
       END AS trend
FROM monthly_totals
ORDER BY month_num;
```

■ *Combine LAG window function with CASE for trend analysis.*

WINDOW FUNCTIONS

•	**Q31. Rank all products by total sales (highest to lowest). Show product ID, total sales, and rank.**

✔ SOLUTION

```
SELECT prod_id,
       ROUND(SUM(sales), 2) AS total_sales,
       RANK() OVER (ORDER BY SUM(sales) DESC) AS sales_rank,
       DENSE_RANK() OVER (ORDER BY SUM(sales) DESC) AS dense_rank
FROM fact_sales
GROUP BY prod_id
ORDER BY sales_rank;
```

■ *RANK() allows gaps with ties; DENSE_RANK() has no gaps.*

•	**Q32. For each sale, show the profit and cumulative profit total ordered by order ID.**

✔ SOLUTION

```
SELECT ord_id, prod_id, profit,
       SUM(profit) OVER (ORDER BY ord_id ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS running_total_profit
FROM fact_sales
ORDER BY ord_id;
```

■ *Use ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW for running totals.*

•	**Q33. Show each product within its category ranked by unit price (highest first).**

✔ **SOLUTION**

```
SELECT category, prod_name, unit_price,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) AS
price_rank
FROM dim_prod
ORDER BY category, price_rank;
```

■ *PARTITION BY resets ranking for each category independently.*

▲ **Q34. Compare each month's order count to previous month using LAG function.**

✔ **SOLUTION**

```
WITH monthly AS (
  SELECT MONTH(order_date) AS monthnumber,
         COUNT(ord_id) AS monthlycount
  FROM dim_order
  GROUP BY MONTH(order_date)
)
SELECT monthnumber, monthlycount,
       LAG(monthlycount, 1) OVER (ORDER BY monthnumber) AS prev_month_count,
       monthlycount - LAG(monthlycount, 1) OVER (ORDER BY monthnumber) AS
month_change
FROM monthly
ORDER BY monthnumber;
```

■ *LAG(col, 1) fetches previous row; offset of 1 = immediately prior.*

▲ **Q35. Show 3-month moving average of sales using window function.**

✔ **SOLUTION**

```
WITH monthly_sales AS (
  SELECT MONTH(d.order_date) AS month_num,
         SUM(f.sales) AS monthly_sales
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY MONTH(d.order_date)
)
SELECT month_num, monthly_sales,
       ROUND(AVG(monthly_sales) OVER (
         ORDER BY month_num
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ), 2) AS moving_avg_3mo
FROM monthly_sales
ORDER BY month_num;
```

■ *Use ROWS BETWEEN 2 PRECEDING for 3-period moving window.*

▲ **Q36. Classify customers into quartiles by total profit (Q1 lowest, Q4 highest).**

✔ **SOLUTION**

```
WITH cust_profit AS (
  SELECT cust_id, SUM(profit) AS total_profit
  FROM fact_sales
  GROUP BY cust_id
)
SELECT cust_id, total_profit,
       NTILE(4) OVER (ORDER BY total_profit DESC) AS quartile,
       CASE
         WHEN NTILE(4) OVER (ORDER BY total_profit DESC) = 1 THEN 'Q4-Top'
         WHEN NTILE(4) OVER (ORDER BY total_profit DESC) = 2 THEN 'Q3'
         WHEN NTILE(4) OVER (ORDER BY total_profit DESC) = 3 THEN 'Q2'
         ELSE 'Q1-Bottom'
       END AS quartile_label
FROM cust_profit
ORDER BY total_profit DESC;
```

■ *NTILE(4) divides rows into 4 equal-sized buckets.*

▲	**Q37. Show row number within each product category ordered by profit (descending).**

✔ **SOLUTION**

```
SELECT category, prod_name, SUM(profit) AS total_profit,
       ROW_NUMBER() OVER (PARTITION BY category ORDER BY SUM(profit) DESC) AS
rn
FROM fact_sales f
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY category, prod_name
ORDER BY category, rn;
```

■ *ROW_NUMBER() always gives unique numbers; PARTITION BY creates per-group numbering.*

▲	**Q38. Show first and last sale date for each customer.**

✔ **SOLUTION**

```
SELECT c.cust_id, c.cust_name,
       MIN(d.order_date) AS first_order_date,
       MAX(d.order_date) AS last_order_date,
       DATEDIFF(MAX(d.order_date), MIN(d.order_date)) AS days_active
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
INNER JOIN dim_order d ON f.ord_id = d.ord_id
GROUP BY c.cust_id, c.cust_name
ORDER BY days_active DESC;
```

■ *MIN/MAX on dates gives first/last; DATEDIFF shows customer lifetime.*

■	**Q39. For each product, show position relative to category average profit.**

✔ SOLUTION

```
WITH cat_avg AS (
  SELECT p.category, AVG(SUM(f.profit)) AS avg_cat_profit
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category, p.prod_id
)
SELECT p.category, p.prod_name,
       SUM(f.profit) AS prod_profit,
       (SELECT avg_cat_profit FROM cat_avg ca WHERE ca.category = p.category
LIMIT 1) AS avg_cat_profit,
       CASE
         WHEN SUM(f.profit) > (SELECT avg_cat_profit FROM cat_avg ca WHERE
ca.category = p.category LIMIT 1)
           THEN 'Above Average'
         ELSE 'Below Average'
       END AS position
FROM fact_sales f
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY p.category, p.prod_name
ORDER BY p.category, SUM(f.profit) DESC;
```

■ *Compare individual against window aggregate using correlated subquery.*

■ Q40. Show sales percentage contribution of each product within its category.

✔ SOLUTION

```
WITH cat_totals AS (
  SELECT p.category,
         p.prod_id,
         SUM(f.sales) AS prod_sales,
         SUM(SUM(f.sales)) OVER (PARTITION BY p.category) AS cat_total_sales
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category, p.prod_id
)
SELECT category, prod_id,
       ROUND(prod_sales, 2) AS prod_sales,
       ROUND(cat_total_sales, 2) AS cat_total,
       ROUND(prod_sales / cat_total_sales * 100, 2) AS pct_of_category
FROM cat_totals
ORDER BY category, pct_of_category DESC;
```

■ *Use window SUM with PARTITION BY to calculate category totals.*

■ Q41. Find top 2 customers per segment by total profit.

✔ SOLUTION

```
WITH cust_seg_profit AS (
```

```
  SELECT c.cust_segment, c.cust_id, c.cust_name,
         SUM(f.profit) AS total_profit,
         ROW_NUMBER() OVER (PARTITION BY c.cust_segment ORDER BY SUM(f.profit)
DESC) AS seg_rank
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  GROUP BY c.cust_segment, c.cust_id, c.cust_name
)
SELECT cust_segment, cust_id, cust_name, ROUND(total_profit, 2) AS total_profit
FROM cust_seg_profit
WHERE seg_rank <= 2
ORDER BY cust_segment, seg_rank;
```

■ *ROW_NUMBER with PARTITION BY and filter on rank.*

■ Q42. Show sales trend: current month vs previous month with growth percentage.

✔ SOLUTION

```
WITH monthly_sales AS (
  SELECT MONTH(d.order_date) AS month_num,
         SUM(f.sales) AS current_sales,
         LAG(SUM(f.sales)) OVER (ORDER BY MONTH(d.order_date)) AS prev_sales
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY MONTH(d.order_date)
)
SELECT month_num,
       ROUND(current_sales, 2) AS current_sales,
       ROUND(prev_sales, 2) AS prev_sales,
       CASE
         WHEN prev_sales IS NULL THEN NULL
         ELSE ROUND((current_sales - prev_sales) / prev_sales * 100, 2)
       END AS growth_pct
FROM monthly_sales
ORDER BY month_num;
```

■ *Calculate percentage change using (current - previous) / previous * 100.*

■ Q43. Show cumulative profit by shipping mode across all orders.

✔ SOLUTION

```
SELECT s.ship_mode, f.ord_id, SUM(f.profit) AS order_profit,
       SUM(SUM(f.profit)) OVER (
         PARTITION BY s.ship_mode
         ORDER BY f.ord_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_profit
FROM fact_sales f
INNER JOIN dim_shipping s ON f.ship_id = s.ship_id
GROUP BY s.ship_mode, f.ord_id
```

```
ORDER BY s.ship_mode, f.ord_id;
```

■ *Nest aggregation: group by ord_id first, then cumulative sum by ship_mode.*

■ Q44. Show each customer's order sequence number and days since previous order.

✔ SOLUTION

```
WITH cust_orders AS (
  SELECT f.cust_id, d.ord_id, d.order_date,
         ROW_NUMBER() OVER (PARTITION BY f.cust_id ORDER BY d.order_date) AS
order_seq,
         LAG(d.order_date) OVER (PARTITION BY f.cust_id ORDER BY d.order_date)
AS prev_order_date
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY f.cust_id, d.ord_id, d.order_date
)
SELECT cust_id, ord_id, order_date, order_seq,
       CASE
         WHEN order_seq = 1 THEN 0
         ELSE DATEDIFF(order_date, prev_order_date)
       END AS days_since_prev_order
FROM cust_orders
ORDER BY cust_id, order_seq;
```

■ *Combine ROW_NUMBER for sequence with LAG for previous date.*

■ Q45. Calculate z-score (deviation from mean) for product profits to identify outliers.

✔ SOLUTION

```
WITH prod_stats AS (
  SELECT prod_id,
         SUM(profit) AS total_profit,
         AVG(SUM(profit)) OVER () AS mean_profit,
         STDDEV(SUM(profit)) OVER () AS stddev_profit
  FROM fact_sales
  GROUP BY prod_id
)
SELECT prod_id, total_profit,
       CASE
         WHEN stddev_profit = 0 THEN 0
         ELSE (total_profit - mean_profit) / stddev_profit
       END AS z_score,
       CASE
         WHEN ABS((total_profit - mean_profit) / NULLIF(stddev_profit, 0)) > 2
THEN 'Outlier'
         ELSE 'Normal'
       END AS classification
FROM prod_stats
```

```
ORDER BY z_score DESC;
```

■ *Use STDDEV window function and divide (value - mean) / stddev.*

COMPLEX COMBINATIONS

▲ Q46. Find customers who purchased from all product categories.

✔ SOLUTION

```
SELECT f.cust_id, COUNT(DISTINCT p.category) AS categories_purchased
FROM fact_sales f
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY f.cust_id
HAVING COUNT(DISTINCT p.category) = (SELECT COUNT(DISTINCT category) FROM
dim_prod)
ORDER BY categories_purchased DESC;
```

■ *Relational division: count distinct categories must equal total categories.*

▲ Q47. Find pairs of customers from the same city.

✔ SOLUTION

```
SELECT a.cust_id AS cust_a, b.cust_id AS cust_b, a.cust_city AS shared_city
FROM dim_cust a
INNER JOIN dim_cust b ON a.cust_city = b.cust_city AND a.cust_id < b.cust_id
ORDER BY shared_city, cust_a;
```

■ *Self-join with a.cust_id < b.cust_id prevents duplicate pairs.*

▲ Q48. Show products never ordered and why (no sales record exists).

✔ SOLUTION

```
SELECT p.prod_id, p.prod_name, p.category
FROM dim_prod p
LEFT JOIN fact_sales f ON p.prod_id = f.prod_id
WHERE f.prod_id IS NULL
ORDER BY p.category;
```

■ *LEFT JOIN + IS NULL finds unmatched records on right side.*

▲ Q49. Show customers with zero orders (exist in dim_cust but not in fact_sales).

✔ SOLUTION

```
SELECT c.cust_id, c.cust_name, c.cust_segment, c.cust_city
FROM dim_cust c
LEFT JOIN fact_sales f ON c.cust_id = f.cust_id
WHERE f.cust_id IS NULL
ORDER BY c.cust_name;
```

■ *LEFT JOIN dim_cust first, then check for NULL in fact_sales.*

▲ **Q50. Display all customers and products with NO overlap (those with zero sales).**

✔ **SOLUTION**

```
SELECT c.cust_id, c.cust_name, NULL AS prod_id
FROM dim_cust c
LEFT JOIN fact_sales f ON c.cust_id = f.cust_id
WHERE f.cust_id IS NULL
UNION
SELECT NULL, NULL, p.prod_id, p.prod_name
FROM dim_prod p
LEFT JOIN fact_sales f ON p.prod_id = f.prod_id
WHERE f.prod_id IS NULL
ORDER BY cust_id;
```

■ *UNION two LEFT JOINs to simulate FULL OUTER JOIN.*

▲ **Q51. Show orders with detailed timeline: order date, ship date, delivery estimate, and actual days shipped.**

✔ **SOLUTION**

```
SELECT d.ord_id, d.order_date, d.ship_date,
       DATEDIFF(d.ship_date, d.order_date) AS actual_ship_days,
       CASE
         WHEN DATEDIFF(d.ship_date, d.order_date) <= 3 THEN 'Express'
         WHEN DATEDIFF(d.ship_date, d.order_date) <= 7 THEN 'Standard'
         ELSE 'Delayed'
       END AS shipping_class,
       COUNT(f.prod_id) AS line_items,
       ROUND(SUM(f.sales), 2) AS order_total
FROM dim_order d
LEFT JOIN fact_sales f ON d.ord_id = f.ord_id
GROUP BY d.ord_id, d.order_date, d.ship_date
ORDER BY actual_ship_days DESC;
```

■ *Combine date functions with aggregation and CASE classification.*

▲ **Q52. Find products in top 3 categories by profit, ranked within category by individual product profit.**

✔ **SOLUTION**

```
WITH top_cats AS (
  SELECT p.category
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category
  ORDER BY SUM(f.profit) DESC
```

```
  LIMIT 3
),
ranked_prods AS (
  SELECT p.category, p.prod_name, SUM(f.profit) AS prod_profit,
         ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(f.profit)
DESC) AS cat_rank
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  WHERE p.category IN (SELECT category FROM top_cats)
  GROUP BY p.category, p.prod_name
)
SELECT category, prod_name, prod_profit, cat_rank
FROM ranked_prods
ORDER BY category, cat_rank;
```

■ *First CTE finds top categories, second ranks products within them.*

▲ Q53. Show month-over-month growth analysis for each product category.

✔ SOLUTION

```
WITH monthly_cat_sales AS (
  SELECT p.category, MONTH(d.order_date) AS month_num, SUM(f.sales) AS
monthly_sales
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.category, MONTH(d.order_date)
)
SELECT category, month_num,
       ROUND(monthly_sales, 2) AS current_sales,
       ROUND(LAG(monthly_sales) OVER (PARTITION BY category ORDER BY
month_num), 2) AS prev_sales,
       CASE
         WHEN LAG(monthly_sales) OVER (PARTITION BY category ORDER BY
month_num) IS NULL THEN NULL
         ELSE ROUND((monthly_sales - LAG(monthly_sales) OVER (PARTITION BY
category ORDER BY month_num))
                    / LAG(monthly_sales) OVER (PARTITION BY category ORDER BY
month_num) * 100, 2)
       END AS growth_pct
FROM monthly_cat_sales
ORDER BY category, month_num;
```

■ *Partition LAG by category to get month-over-month within each category.*

▲ Q54. Compare order size distribution: show count of orders in size brackets (small <100, medium 100-500, large >500).

✔ SOLUTION

```
WITH order_sizes AS (
  SELECT d.ord_id, SUM(f.sales) AS order_amount
```

```
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY d.ord_id
)
SELECT
  CASE
    WHEN order_amount < 100 THEN 'Small (<100)'
    WHEN order_amount <= 500 THEN 'Medium (100-500)'
    ELSE 'Large (>500)'
  END AS order_size,
  COUNT(*) AS order_count,
  ROUND(AVG(order_amount), 2) AS avg_amount,
  ROUND(MIN(order_amount), 2) AS min_amount,
  ROUND(MAX(order_amount), 2) AS max_amount
FROM order_sizes
GROUP BY order_size
ORDER BY avg_amount;
```

■ *CTE aggregates to order level, then outer query creates size brackets.*

▲ Q55. Show customer lifetime value (LTV) with profitability breakdown.

✔ SOLUTION

```
SELECT c.cust_id, c.cust_name, c.cust_segment,
       COUNT(DISTINCT f.ord_id) AS total_orders,
       ROUND(SUM(f.sales), 2) AS lifetime_sales,
       ROUND(SUM(f.profit), 2) AS lifetime_profit,
       ROUND(SUM(f.profit) / SUM(f.sales) * 100, 2) AS profit_margin_pct,
       ROUND(SUM(f.profit) / COUNT(DISTINCT f.ord_id), 2) AS profit_per_order
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
GROUP BY c.cust_id, c.cust_name, c.cust_segment
ORDER BY lifetime_profit DESC;
```

■ *Calculate LTV and breakdown metrics for customer segmentation.*

▲ Q56. Find products that appear in more orders than the median product order count.

✔ SOLUTION

```
WITH prod_orders AS (
  SELECT prod_id, COUNT(DISTINCT ord_id) AS order_count
  FROM fact_sales
  GROUP BY prod_id
)
SELECT prod_id, order_count
FROM prod_orders
WHERE order_count > (
  SELECT MEDIAN(order_count)
  FROM (SELECT COUNT(DISTINCT ord_id) AS order_count FROM fact_sales GROUP BY
prod_id) sub
```

```
)
ORDER BY order_count DESC;
```

■ *Use PERCENTILE_CONT(0.5) if MEDIAN not available in your MySQL version.*

▲ Q57. Show top 5 customer-product combinations by sales volume.

✔ SOLUTION

```
SELECT c.cust_id, c.cust_name, p.prod_id, p.prod_name,
       COUNT(f.ord_id) AS transaction_count,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(AVG(f.sales), 2) AS avg_transaction
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY c.cust_id, c.cust_name, p.prod_id, p.prod_name
ORDER BY total_sales DESC
LIMIT 5;
```

■ *Group by both customer and product for combination analysis.*

■ Q58. Create customer segmentation: RFM analysis (Recency, Frequency, Monetary).

✔ SOLUTION

```
WITH cust_rfm AS (
  SELECT c.cust_id, c.cust_name,
         DATEDIFF((SELECT MAX(order_date) FROM dim_order), MAX(d.order_date))
AS recency_days,
         COUNT(DISTINCT f.ord_id) AS frequency,
         ROUND(SUM(f.sales), 2) AS monetary_value
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY c.cust_id, c.cust_name
)
SELECT cust_id, cust_name, recency_days, frequency, monetary_value,
       CASE
         WHEN recency_days <= 30 AND frequency > 5 AND monetary_value > 1000
THEN 'VIP'
         WHEN recency_days <= 90 AND frequency > 3 THEN 'High Value'
         WHEN recency_days > 180 THEN 'At Risk'
         ELSE 'Regular'
       END AS segment
FROM cust_rfm
ORDER BY monetary_value DESC;
```

■ *Calculate R (days since last order), F (order count), M (total spend).*

■ Q59. Analyze product performance: sales trend, profit margin, and rank vs category average.

✔ SOLUTION

```
WITH prod_metrics AS (
  SELECT p.prod_id, p.prod_name, p.category,
         SUM(f.sales) AS total_sales,
         SUM(f.profit) AS total_profit,
         SUM(f.profit) / NULLIF(SUM(f.sales), 0) * 100 AS margin_pct,
         COUNT(DISTINCT f.ord_id) AS order_count,
         AVG(SUM(f.sales)) OVER (PARTITION BY p.category) AS cat_avg_sales
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.prod_id, p.prod_name, p.category
)
SELECT prod_id, prod_name, category,
       ROUND(total_sales, 2) AS total_sales,
       ROUND(total_profit, 2) AS total_profit,
       ROUND(margin_pct, 2) AS margin_pct,
       order_count,
       ROUND(cat_avg_sales, 2) AS category_avg,
       CASE
         WHEN total_sales > cat_avg_sales THEN 'Above Avg'
         ELSE 'Below Avg'
       END AS performance
FROM prod_metrics
ORDER BY category, total_sales DESC;
```

■ *Window function calculates category average for comparison.*

■ Q60. Show shipping mode analysis with cost-benefit: average profit vs shipping days.

✔ SOLUTION

```
SELECT s.ship_mode,
       COUNT(DISTINCT f.ord_id) AS total_orders,
       ROUND(AVG(DATEDIFF(d.ship_date, d.order_date)), 2) AS avg_ship_days,
       ROUND(AVG(f.profit), 2) AS avg_profit,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.profit) / COUNT(DISTINCT f.ord_id), 2) AS profit_per_order
FROM fact_sales f
INNER JOIN dim_shipping s ON f.ship_id = s.ship_id
INNER JOIN dim_order d ON f.ord_id = d.ord_id
GROUP BY s.ship_mode
ORDER BY avg_profit DESC;
```

■ *Compare shipping modes on both speed and profitability.*

■ Q61. Build customer journey: first purchase date, last purchase date, total value, and repeat rate.

✔ SOLUTION

```sql
WITH cust_journey AS (
  SELECT c.cust_id, c.cust_name,
         MIN(d.order_date) AS first_purchase,
         MAX(d.order_date) AS last_purchase,
         DATEDIFF(MAX(d.order_date), MIN(d.order_date)) AS days_active,
         COUNT(DISTINCT f.ord_id) AS total_orders,
         ROUND(SUM(f.sales), 2) AS total_value,
         ROUND(SUM(f.profit), 2) AS total_profit
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY c.cust_id, c.cust_name
)
SELECT cust_id, cust_name, first_purchase, last_purchase, days_active,
total_orders,
       total_value, total_profit,
       CASE
         WHEN total_orders = 1 THEN 'One-time'
         ELSE 'Repeat'
       END AS customer_type
FROM cust_journey
ORDER BY total_value DESC;
```

■ *Combine date MIN/MAX with COUNT for complete journey view.*

■ Q62. Show sales seasonality: analyze each month's sales, profit, and contribution to annual total.

✔ SOLUTION

```sql
WITH monthly_analysis AS (
  SELECT MONTH(d.order_date) AS month_num,
         SUM(f.sales) AS monthly_sales,
         SUM(f.profit) AS monthly_profit,
         COUNT(DISTINCT f.ord_id) AS order_count,
         COUNT(DISTINCT f.cust_id) AS customer_count
  FROM fact_sales f
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY MONTH(d.order_date)
)
SELECT month_num,
       ROUND(monthly_sales, 2) AS monthly_sales,
       ROUND(monthly_profit, 2) AS monthly_profit,
       order_count, customer_count,
       ROUND(monthly_sales / (SELECT SUM(monthly_sales) FROM monthly_analysis)
* 100, 2) AS pct_annual_sales
FROM monthly_analysis
ORDER BY month_num;
```

■ *Calculate both absolute metrics and percentage contribution.*

■ Q63. Analyze discount impact: compare metrics for discounted vs non-discounted sales.

✔ **SOLUTION**

```
SELECT
       CASE
         WHEN discount = 0 THEN 'No Discount'
         ELSE 'Discounted'
       END AS discount_status,
       COUNT(*) AS transaction_count,
       ROUND(AVG(sales), 2) AS avg_sales,
       ROUND(AVG(profit), 2) AS avg_profit,
       ROUND(SUM(sales), 2) AS total_sales,
       ROUND(SUM(profit), 2) AS total_profit,
       ROUND(SUM(profit) / SUM(sales) * 100, 2) AS margin_pct,
       ROUND(AVG(discount), 4) AS avg_discount_rate
FROM fact_sales
GROUP BY discount_status
ORDER BY transaction_count DESC;
```

■ *Compare profitability metrics between discounted and non-discounted.*

■ **Q64. Show product portfolio maturity: new (low order count), mature (high orders, stable margin), declining (decreasing sales).**

✔ **SOLUTION**

```
WITH prod_analysis AS (
  SELECT p.prod_id, p.prod_name,
         COUNT(DISTINCT f.ord_id) AS order_count,
         SUM(f.sales) AS total_sales,
         SUM(f.profit) / NULLIF(SUM(f.sales), 0) * 100 AS margin_pct
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.prod_id, p.prod_name
)
SELECT prod_id, prod_name, order_count, ROUND(total_sales, 2) AS total_sales,
       ROUND(margin_pct, 2) AS margin_pct,
       CASE
         WHEN order_count < 5 THEN 'New'
         WHEN order_count >= 10 AND margin_pct > 15 THEN 'Mature'
         WHEN order_count >= 5 AND margin_pct < 10 THEN 'Declining'
         ELSE 'Growing'
       END AS lifecycle_stage
FROM prod_analysis
ORDER BY order_count DESC;
```

■ *Use order count and margin thresholds to classify product maturity.*

■ **Q65. Build a complete sales report: by segment, category, and geography (customer city).**

✔ **SOLUTION**

```
SELECT c.cust_segment, p.category, c.cust_city,
       COUNT(DISTINCT c.cust_id) AS customer_count,
       COUNT(DISTINCT f.ord_id) AS order_count,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(AVG(f.sales), 2) AS avg_order_value,
       ROUND(SUM(f.profit) / SUM(f.sales) * 100, 2) AS profit_margin_pct
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY c.cust_segment, p.category, c.cust_city
ORDER BY total_profit DESC;
```

■ *Triple aggregation: by segment, category, and city simultaneously.*

■ Q66. Identify product cross-sell opportunities: products frequently bought together.

✔ SOLUTION

```
SELECT f1.prod_id AS prod_a, f2.prod_id AS prod_b,
       COUNT(DISTINCT f1.ord_id) AS co_purchase_count
FROM fact_sales f1
INNER JOIN fact_sales f2 ON f1.ord_id = f2.ord_id AND f1.prod_id < f2.prod_id
GROUP BY f1.prod_id, f2.prod_id
HAVING COUNT(DISTINCT f1.ord_id) >= 3
ORDER BY co_purchase_count DESC
LIMIT 10;
```

■ *Self-join fact_sales on order ID to find products in same order.*

■ Q67. Show customer acquisition cost proxy: new customers by month and their initial transaction value.

✔ SOLUTION

```
WITH first_purchases AS (
  SELECT c.cust_id, c.cust_segment,
         MIN(d.order_date) AS first_order_date,
         MIN(f.sales) AS first_order_value
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY c.cust_id, c.cust_segment
)
SELECT MONTH(first_order_date) AS month_acquired,
       COUNT(*) AS new_customers,
       ROUND(AVG(first_order_value), 2) AS avg_first_order_value,
       ROUND(SUM(first_order_value), 2) AS total_new_customer_sales
FROM first_purchases
GROUP BY MONTH(first_order_date)
ORDER BY month_acquired;
```

■ *Use MIN(order_date) to identify first purchase for each customer.*

■ Q68. Calculate customer churn risk: customers inactive for >90 days in current analysis period.

✔ SOLUTION

```
WITH cust_activity AS (
  SELECT c.cust_id, c.cust_name, c.cust_segment,
         MAX(d.order_date) AS last_order_date,
         DATEDIFF((SELECT MAX(order_date) FROM dim_order), MAX(d.order_date))
AS days_inactive,
         COUNT(DISTINCT f.ord_id) AS total_orders,
         SUM(f.profit) AS total_profit
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY c.cust_id, c.cust_name, c.cust_segment
)
SELECT cust_id, cust_name, cust_segment, last_order_date, days_inactive,
total_orders,
       ROUND(total_profit, 2) AS total_profit,
       CASE
         WHEN days_inactive > 180 THEN 'High Risk'
         WHEN days_inactive > 90 THEN 'Medium Risk'
         ELSE 'Active'
       END AS churn_risk
FROM cust_activity
ORDER BY days_inactive DESC;
```

■ *Use MAX date and DATEDIFF to calculate inactivity period.*

■ Q69. Show market basket analysis: category affinity (products from same category in orders).

✔ SOLUTION

```
WITH order_cats AS (
  SELECT DISTINCT f.ord_id, p.category
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
),
cat_pairs AS (
  SELECT oc1.ord_id, oc1.category AS cat1, oc2.category AS cat2
  FROM order_cats oc1
  INNER JOIN order_cats oc2 ON oc1.ord_id = oc2.ord_id AND oc1.category <
oc2.category
)
SELECT cat1, cat2, COUNT(*) AS co_occurrence,
       ROUND(COUNT(*) / (SELECT COUNT(DISTINCT ord_id) FROM order_cats) * 100,
2) AS pct_orders
FROM cat_pairs
```

```
GROUP BY cat1, cat2
ORDER BY co_occurrence DESC;
```

■ *Find which product categories appear together in orders.*

■ Q70. Comprehensive 360 customer view: all metrics combined for top 10 customers by profit.

✔ SOLUTION

```
WITH cust_360 AS (
  SELECT c.cust_id, c.cust_name, c.cust_segment, c.cust_city,
         COUNT(DISTINCT f.ord_id) AS total_orders,
         COUNT(DISTINCT p.category) AS categories_purchased,
         MIN(d.order_date) AS first_order_date,
         MAX(d.order_date) AS last_order_date,
         ROUND(SUM(f.sales), 2) AS lifetime_sales,
         ROUND(SUM(f.profit), 2) AS lifetime_profit,
         ROUND(AVG(f.sales), 2) AS avg_order_value,
         ROUND(SUM(f.profit) / SUM(f.sales) * 100, 2) AS profit_margin,
         DENSE_RANK() OVER (ORDER BY SUM(f.profit) DESC) AS profit_rank
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  INNER JOIN dim_order d ON f.ord_id = d.ord_id
  GROUP BY c.cust_id, c.cust_name, c.cust_segment, c.cust_city
)
SELECT * FROM cust_360 WHERE profit_rank <= 10 ORDER BY profit_rank;
```

■ *Multi-CTE with all key metrics: recency, frequency, monetary, category diversity, rank.*

■ Q71. Show product performance scorecard: rank products across multiple dimensions (sales, profit, margin, orders).

✔ SOLUTION

```
WITH scoring AS (
  SELECT p.prod_id, p.prod_name, p.category,
         SUM(f.sales) AS total_sales,
         SUM(f.profit) AS total_profit,
         COUNT(DISTINCT f.ord_id) AS order_count,
         SUM(f.profit) / NULLIF(SUM(f.sales), 0) AS margin_ratio,
         RANK() OVER (ORDER BY SUM(f.sales) DESC) AS sales_rank,
         RANK() OVER (ORDER BY SUM(f.profit) DESC) AS profit_rank,
         RANK() OVER (ORDER BY COUNT(DISTINCT f.ord_id) DESC) AS order_rank
  FROM fact_sales f
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY p.prod_id, p.prod_name, p.category
)
SELECT prod_id, prod_name, category,
       ROUND(total_sales, 2) AS total_sales, sales_rank,
       ROUND(total_profit, 2) AS total_profit, profit_rank,
       order_count, order_rank,
```

```
       ROUND(margin_ratio * 100, 2) AS margin_pct,
       (sales_rank + profit_rank + order_rank) / 3 AS avg_rank_score
FROM scoring
ORDER BY avg_rank_score
LIMIT 20;
```

■ *Rank on multiple metrics, then average ranks for composite score.*

■ Q72. Analyze shipping efficiency: compare actual vs desired days, cost per day effectiveness.

✔ SOLUTION

```
SELECT s.ship_mode,
       COUNT(DISTINCT f.ord_id) AS orders,
       ROUND(AVG(DATEDIFF(d.ship_date, d.order_date)), 2) AS avg_ship_days,
       CASE
         WHEN AVG(DATEDIFF(d.ship_date, d.order_date)) <= 3 THEN 'Excellent'
         WHEN AVG(DATEDIFF(d.ship_date, d.order_date)) <= 7 THEN 'Good'
         ELSE 'Slow'
       END AS efficiency_rating,
       ROUND(AVG(f.sales), 2) AS avg_order_value,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.profit) / COUNT(DISTINCT f.ord_id), 2) AS profit_per_order
FROM fact_sales f
INNER JOIN dim_shipping s ON f.ship_id = s.ship_id
INNER JOIN dim_order d ON f.ord_id = d.ord_id
GROUP BY s.ship_mode
ORDER BY avg_ship_days;
```

■ *Combine date metrics with profitability for efficiency analysis.*

■ Q73. Show category-specific customer segments: which segments buy which categories most.

✔ SOLUTION

```
WITH seg_cat_sales AS (
  SELECT c.cust_segment, p.category,
         COUNT(DISTINCT c.cust_id) AS customer_count,
         SUM(f.sales) AS segment_cat_sales,
         SUM(SUM(f.sales)) OVER (PARTITION BY c.cust_segment) AS
segment_total_sales
  FROM fact_sales f
  INNER JOIN dim_cust c ON f.cust_id = c.cust_id
  INNER JOIN dim_prod p ON f.prod_id = p.prod_id
  GROUP BY c.cust_segment, p.category
)
SELECT cust_segment, category, customer_count,
       ROUND(segment_cat_sales, 2) AS category_sales,
       ROUND(segment_total_sales, 2) AS segment_total,
       ROUND(segment_cat_sales / segment_total_sales * 100, 2) AS
pct_of_segment_sales
```

```
FROM seg_cat_sales
ORDER BY cust_segment, pct_of_segment_sales DESC;
```

■ *Window SUM to calculate segment totals, then derive percentages.*

■ Q74. Find best customer-segment-product combinations for focused marketing.

✔ SOLUTION

```
SELECT c.cust_segment, p.category, COUNT(DISTINCT c.cust_id) AS customers,
       COUNT(DISTINCT f.ord_id) AS orders,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.profit) / SUM(f.sales) * 100, 2) AS profit_margin,
       ROUND(AVG(f.sales), 2) AS avg_transaction,
       DENSE_RANK() OVER (ORDER BY SUM(f.profit) DESC) AS profit_rank
FROM fact_sales f
INNER JOIN dim_cust c ON f.cust_id = c.cust_id
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY c.cust_segment, p.category
HAVING COUNT(DISTINCT f.ord_id) >= 3
ORDER BY profit_rank
LIMIT 15;
```

■ *Triple dimension analysis with HAVING filter for minimum activity.*

■ Q75. Ultimate dashboard summary: key KPIs by month (sales, profit, orders, customers, avg order value).

✔ SOLUTION

```
SELECT MONTH(d.order_date) AS month_num,
       COUNT(DISTINCT f.ord_id) AS total_orders,
       COUNT(DISTINCT f.cust_id) AS unique_customers,
       COUNT(DISTINCT p.prod_id) AS unique_products,
       ROUND(SUM(f.sales), 2) AS total_sales,
       ROUND(SUM(f.profit), 2) AS total_profit,
       ROUND(SUM(f.profit) / SUM(f.sales) * 100, 2) AS profit_margin_pct,
       ROUND(SUM(f.sales) / COUNT(DISTINCT f.ord_id), 2) AS avg_order_value,
       ROUND(SUM(f.sales) / COUNT(DISTINCT f.cust_id), 2) AS
avg_customer_value,
       ROUND(AVG(DATEDIFF(d.ship_date, d.order_date)), 2) AS avg_ship_days
FROM fact_sales f
INNER JOIN dim_order d ON f.ord_id = d.ord_id
INNER JOIN dim_prod p ON f.prod_id = p.prod_id
GROUP BY MONTH(d.order_date)
ORDER BY month_num;
```

■ *Combine all major KPIs for executive-level dashboard at monthly granularity.*

Database Schema Reference

- fact_sales: ord_id, cust_id, prod_id, ship_id, sales, profit, discount
- dim_cust: cust_id, cust_name, cust_segment, cust_city
- dim_order: ord_id, order_date, ship_date
- dim_prod: prod_id, prod_name, category, unit_price
- dim_shipping: ship_id, ship_mode